

BÁO CÁO KỸ THUẬT & KIỂM THỬ TẢI

BÀI TẬP 3: CHUYỂN ĐỔI RESTTEMPLATE SANG WEBCLIENT TRONG SPRING WEBFLUX

Khôi phục kiến trúc Non-blocking, giải cứu API khỏi cạn kiệt luồng khi phục vụ 10.000 concurrent users tại StoreX

Khóa học: Microservices System Design (Rikkei Education)	Hệ thống: Nền tảng E-Commerce StoreX
Module: storex-home-service → promotion-service	Công nghệ: Spring WebFlux, Project Reactor (Mono), Netty, WebClient, StepVerifier

1. Bối cảnh Nghiệp vụ & Hiện trạng Sự cố

Hệ thống **StoreX** cung cấp API hiển thị **Banner Khuyến mãi** tại trang chủ. Khi khách hàng mở ứng dụng, API này gọi sang **promotion-service** để lấy thông tin khuyến mãi đang hiệu lực. Mục tiêu phi chức năng là hệ thống phải chịu tải mượt mà **10.000 người dùng truy cập đồng thời** trong các khung giờ Flash Sale.

Dự án được cấu hình chạy trên nền tảng **Spring WebFlux**. Tuy nhiên, khi chạy kiểm thử chịu tải, hệ thống lập tức xuất hiện hàng loạt lỗi **HTTP 500 Internal Server Error**, server bị "đóng băng", thời gian phản hồi (Response Time) tăng vọt và ứng dụng treo cứng.

```
// PromotionService.java -- HIỆN TRẠNG ĐANG GẶP LỖI
@Service
public class PromotionService {
    // SAI 1: Dùng RestTemplate trong WebFlux -- RestTemplate là BLOCKING
    private final RestTemplate restTemplate = new RestTemplate();

    public Banner getActiveBanner() {
        // SAI 2: Gọi đồng bộ, block thread
        String url = "http://promotion-service/api/banners/active";
        Banner banner = restTemplate.getForObject(url, Banner.class);

        // SAI 3: Nếu service chết, hệ thống crash
        if (banner == null) {
            throw new RuntimeException("Promotion service unavailable");
        }
        return banner;
    }
}
```

2. Phần 1 - Phân Tích Chuyên Sâu Lỗi Kiến Trúc

2.1. Cơ chế hoạt động của RestTemplate & Nguyên nhân gây cạn kiệt Thread

- Mô hình Chặn Đồng bộ (Synchronous Blocking I/O):** **RestTemplate** hoạt động trên mô hình Servlet truyền thống. Khi phương thức **getForObject()** được gọi, luồng (thread) đang xử lý sẽ phát tín hiệu gửi socket qua mạng và ngay lập tức bị đưa vào trạng thái **WAITING / BLOCKED**.

- Lũồng này bị khóa cứng tại chỗ, không thể thực thi thêm bất kỳ tác vụ nào khác cho đến khi nhận được toàn bộ gói tin phản hồi từ `promotion-service`.
- Trong mô hình Tomcat Servlet truyền thống (Thread-per-Request), mỗi request chiếm 1 worker thread. Để phục vụ 10.000 concurrent users, hệ thống cần hàng ngàn thread, dẫn đến chi phí chuyển ngữ cảnh (Context Switching) cực lớn và cạn kiệt bộ nhớ RAM (Stack Memory: mỗi thread ~1MB RAM).

2.2. Tại sao WebFlux không thể xử lý Non-blocking khi có RestTemplate?

Nguyên nhân cốt tử: Vi phạm quy tắc bất biến "NEVER BLOCK THE EVENT LOOP" của Netty!

- **Mô hình Reactor Netty EventLoop:** Spring WebFlux chạy trên máy chủ Netty theo mô hình Event-Driven Non-blocking. Netty không sử dụng hàng trăm thread như Tomcat mà chỉ duy trì một số lượng thread rất nhỏ:
Số Netty EventLoop Threads = 2 × Số CPU Cores
(Ví dụ: Máy chủ có 4 CPU Cores thì Netty chỉ cấp đúng **8 Worker Threads**).
- Các EventLoop thread này dùng cơ chế Non-blocking Selector để phục vụ hàng chục ngàn kết nối đồng thời. Khi request phát sinh I/O mạng, thread Netty không chờ mà ngay lập tức quay vòng phục vụ các request khác.
- **Hiệu ứng "Đầu độc EventLoop" (EventLoop Poisoning):** Khi gọi `restTemplate.getForObject(...)`, lệnh này chạy trực tiếp trên 1 trong 8 EventLoop thread của Netty, khiến thread đó bị **khóa cứng (Blocked)**.
- **Hậu quả khi chịu tải 10.000 users:** Chỉ cần **8 request đầu tiên** gọi RestTemplate và `promotion-service` bị chậm khoảng 1-2 giây, **toàn bộ 8 Netty EventLoop Threads đều bị khóa 100%**. Netty không còn bất kỳ thread nào để tiếp nhận hay điều phối sự kiện I/O cho 9.992 request còn lại. Toàn bộ ứng dụng StoreX bị **"đóng băng" hoàn toàn**, các kết nối bị từ chối hoặc timeout, dẫn đến báo lỗi **HTTP 500 / 504** và sập hệ thống.

2.3. Bảng đối chiếu kiến trúc giữa RestTemplate và WebClient

Tiêu chí so sánh	RestTemplate (Mô hình lỗi)	WebClient (Chuẩn WebFlux Reactive)
Cơ chế I/O	Synchronous Blocking I/O	Asynchronous Non-blocking I/O
Mô hình quản lý Thread	Khóa cứng thread chờ socket (Thread-per-Request)	Sử dụng EventLoop xoay vòng (Netty), không bao giờ block
Số lượng Thread cần cho 10.000 users	Cần hàng ngàn thread (Gây sập server, OOM)	Chỉ cần 4 - 8 EventLoop threads vẫn xử lý mượt mà
Kiểu dữ liệu phản hồi	Đối tượng đồng bộ tĩnh: Banner	Luồng dữ liệu phản ứng bất đồng bộ: Mono<Banner>
Kiểm soát Timeout	Khó kiểm soát, dễ treo vĩnh viễn	Cấu hình tường minh qua toán tử .timeout(Duration.ofSeconds(2))
Cơ chế Phục hồi (Fallback)	Quăng RuntimeException làm crash luồng	Bắt lỗi mượt mà qua .onErrorResume(...) trả về Banner mặc định

3. Phần 2 - Mã Nguồn Sửa Đổi Chuẩn Reactive

3.1. Cấu hình WebClient với Timeout Socket và HTTP (`WebClientConfig.java`)

```
@Configuration
public class WebClientConfig {

    @Bean
    @LoadBalanced
    public WebClient.Builder loadBalancedWebClientBuilder() {
        HttpClient httpClient = HttpClient.create()
            .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 2000) // connectTimeout = 2s
            .responseTimeout(Duration.ofSeconds(2)) // responseTimeout = 2s
            .doOnConnected(conn -> conn
                .addHandlerLast(new ReadTimeoutHandler(2, TimeUnit.SECONDS))
                .addHandlerLast(new WriteTimeoutHandler(2, TimeUnit.SECONDS)));

        return WebClient.builder()
            .clientConnector(new ReactorClientHttpConnector(httpClient));
    }

    @Bean
    public WebClient promotionWebClient(
        WebClient.Builder webClientBuilder,
        @Value("${services.promotion-service.url:http://promotion-service}") String baseUrl
    ) {
        return webClientBuilder.baseUrl(baseUrl).build();
    }
}
```

3.2. Lớp Dịch Vụ Reactive Đã Sửa Đổi (`PromotionService.java`)

```
@Service
@Slf4j
public class PromotionService {

    private final WebClient webClient;
    private static final Duration TIMEOUT_DURATION = Duration.ofSeconds(2);

    @Autowired
    public PromotionService(WebClient webClient) {
        this.webClient = webClient;
    }

    public Mono<Banner> getActiveBanner() {
        log.info("Gửi yêu cầu Non-blocking GET tới /api/banners/active qua WebClient");

        return webClient.get()
            .uri("/api/banners/active")
            .retrieve()
            .bodyToMono(Banner.class)
            // Cấu hình Timeout 2s theo yêu cầu bài toán
            .timeout(TIMEOUT_DURATION)
            .doOnSuccess(banner -> {
                if (banner != null) {
                    log.info("Nhận thành công banner: {}", banner.getTitle());
                }
            })
            .doOnError(error -> log.warn("Sự cố gọi promotion-service: {}. Kích hoạt Fallback.", error.getMessage()))
            // Bắt timeout hoặc lỗi mạng → trả về banner mặc định
            .onErrorResume(error -> Mono.just(Banner.defaultBanner()))
            .defaultIfEmpty(Banner.defaultBanner());
    }
}
```

3.3. Đối Tượng Model Dữ Phòng (Banner.java)

```
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Banner {
    private Long id;
    private String title;
    private String content;
    private String imageUrl;
    private String targetUrl;
    private String status;

    public static Banner defaultBanner() {
        return Banner.builder()
            .id(0L)
            .title("Thông Báo Khuyến Mãi")
            .content("Khuyến mãi đang được cập nhật") // Đúng yêu cầu đề bài
            .imageUrl("/images/banners/default-banner.png")
            .targetUrl("/")
            .status("UPDATING")
            .build();
    }
}
```

4. Kết Quả Kiểm Thử Đơn Vị (Unit Test) Với StepVerifier

Kiểm thử hành vi Reactive bằng thư viện chuẩn `reactor-test` (`StepVerifier`):

Kịch bản Kiểm thử	Mục tiêu & Dữ liệu đầu vào	Kết quả mong đợi	Trạng thái
<code>testGetActiveBanner_Success</code>	Mô phỏng <code>promotion-service</code> trả về Banner thật (Flash Sale 9.9).	<code>StepVerifier</code> nhận đúng Banner với status ACTIVE, kết thúc luồng thành công.	PASSED ✓
<code>testGetActiveBanner_Timeout_ReturnsDefaultBanner</code>	Mô phỏng máy chủ phản hồi trễ quá 2 giây (quảng <code>TimeoutException</code>).	Hệ thống không crash, trả về Banner mặc định với thông báo "Khuyến mãi đang được cập nhật".	PASSED ✓
<code>testGetActiveBanner_ServiceError_ReturnsDefaultBanner</code>	Mô phỏng lỗi HTTP 503 Service Unavailable từ <code>promotion-service</code> .	Bắt lỗi mạng an toàn và kích hoạt Fallback trả về Banner mặc định.	PASSED ✓
<code>testGetActiveBanner_EmptyBody_ReturnsDefaultBanner</code>	Mô phỏng máy chủ trả về body rỗng	Tự động điền Banner mặc định	PASSED ✓

Kịch bản Kiểm thử	Mục tiêu & Dữ liệu đầu vào	Kết quả mong đợi	Trạng thái
	(<code>Mono.empty()</code>).	qua toán tử <code>defaultIfEmpty</code> .	

Xác nhận kết quả: 100% các bài test đều vượt qua (Build SUCCESS). Ứng dụng hoạt động hoàn toàn Non-blocking và miễn nhiễm trước tình trạng cạn kiệt luồng Netty.

5. Kết Luận

Việc thay thế `RestTemplate` bằng `WebClient` và trả về `Mono<Banner>` đã giải quyết triệt để lỗi kiến trúc trong Spring WebFlux:

- Bảo vệ nguyên vẹn các Netty EventLoop Worker Threads, giải phóng luồng ngay lập tức để phục vụ tối đa 10.000 concurrent users.
- Thiết lập trần thời gian phản hồi (Timeout 2s) và cơ chế suy thoái mượt mà (Graceful Degradation), bảo đảm trải nghiệm người dùng không bao giờ bị gián đoạn.